

Security Whitepaper: DROS — Deterministic Execution Trust Boundary Layer

Version 1.0 (Enterprise Specification Grade)

1. Executive Summary

DROS defines a deterministic execution enforcement layer for AI agent systems. It eliminates runtime ambiguity in AI authorization by replacing probabilistic LLM-based decisions, runtime JSON policy engines, and vendor-controlled IAM systems with: > [!IMPORTANT] > **“Compile-time resolved, cryptographically signed execution contracts enforced via O(1) bitmap evaluation.”**

This document details the threat model, mitigation mechanisms, architectural specifications, and compliance alignment of DROS for CISO teams, architecture boards, and procurement evaluators.

2. Problem Statement

Modern enterprise AI systems face three systemic security risks:

2.1 Non-Deterministic Authorization

LLM-based guardrails produce inconsistent decisions under identical inputs. Dynamic semantic boundaries are susceptible to prompt hijacking, roleplay jailbreaks, and neural hallucination.

2.2 Cross-Vendor Policy Fragmentation

Each AI provider (OpenAI, Anthropic, cloud Hyperscalers) defines its own proprietary IAM models, tool permission schemas, and audit log formats. There is no unified, portable execution security standard.

2.3 Audit Non-Reproducibility

Traditional cloud logs record events chronologically, but lack decision provenance. They cannot answer: > *“Why was this action allowed at this specific microsecond?”*

3. Threat Model & Mitigation Matrix

3.1 Prompt Injection Attacks

- **Threat:** Malicious user input manipulates the AI agent's internal prompt history to bypass tool limits (e.g. escalating read permissions to file deletion commands).
- **DROS Mitigation:** No runtime string parsing or dynamic text interpretation occurs during authorization. Permissions are compiled into an immutable bitmap. Since the gateway only performs bitwise operations, prompt vectors are physically non-applicable at the enforcement layer.

3.2 Time-of-Check / Time-of-Use (TOCTOU) Exploits

- **Threat:** Policies evaluated in memory at the user-space runtime are modified by a compromised thread immediately before execution.
- **DROS Mitigation:** Policies are resolved and finalized at compile-time. The resulting `policy.bin` payload is mounted as a Read-Only OS memory map (mmap), preventing in-memory alteration before the FFI C-ABI gate transition.

3.3 Vendor Log Manipulation Risk

- **Threat:** Infrastructure logs are retroactively altered or deleted by a compromised cloud provider or administrative account.
- **DROS Mitigation:** DROS embeds the unique compile-time SHA-256 Policy Hash and Compiler ID directly into the structure of every single audit trace payload, ensuring tamper-proof, reproducible auditing.

4. Security Architecture

[Vajra DSL Source] [Lexicographical Sorting & Compile]

(Ed25519 Signing)

[policy.bin] (Mounted Read-Only)

(Bitwise AND Gate)

[Tool Call Request] [C-ABI Microkernel Interceptor] Hardware Exceptions / Allow

4.1 Compile-Time Trust Model

Policies written in Vajra DSL are compiled by: * Sorting all entities (Agents, Tools, Capabilities) lexicographically to guarantee stable hash outputs. * Signing the binary output using Ed25519 cryptography to guarantee integrity. * Producing an immutable, signed `policy.bin`.

4.2 Runtime Enforcement Model

The FFI runtime microkernel executes permission gates using only: * Constant-time index lookup for Agent and Tool IDs. * Contiguous bitmap matrix evaluation. * $\mathcal{O}(1)$ **complexity** execution bounds.

4.3 Fail-Closed Guarantee

The microkernel enforces strict deny-by-default behavior. Any of the following anomalies instantly trigger a hard thread exception (reverting the boundary to a fail-closed status): * Unknown Agent ID or Tool name. * Policy signature validation mismatch. * Corrupted policy binary checksums.

5. Auditability & Compliance Model

Each execution trace payload generated by the microkernel contains: * `policy_hash` (SHA-256 policy signature). * `compiler_version` and `dsl_version`. * The absolute boolean decision (ALLOW / DENY).

This mathematical binding guarantees **forensic reproducibility** of every single tool execution decision, satisfying compliance audits for: * **ISO/IEC 27001 (Control A.8 & A.9)**. * **SOC 2 Type II**. * **NIST AI Risk Management Framework (AI RMF)**.

6. Deployment Topologies

DROS is designed to operate without any external API calls, cloud dependencies, or dynamic internet lookups. Supported environments include: * Kubernetes/Containerized namespaces (ReadOnly ConfigMaps). * Air-Gapped corporate private clouds. * Mobile edge environments (via compiled Gomobile AAR libraries). * Local embedded hardware loops (via native Zig/Go shared objects).